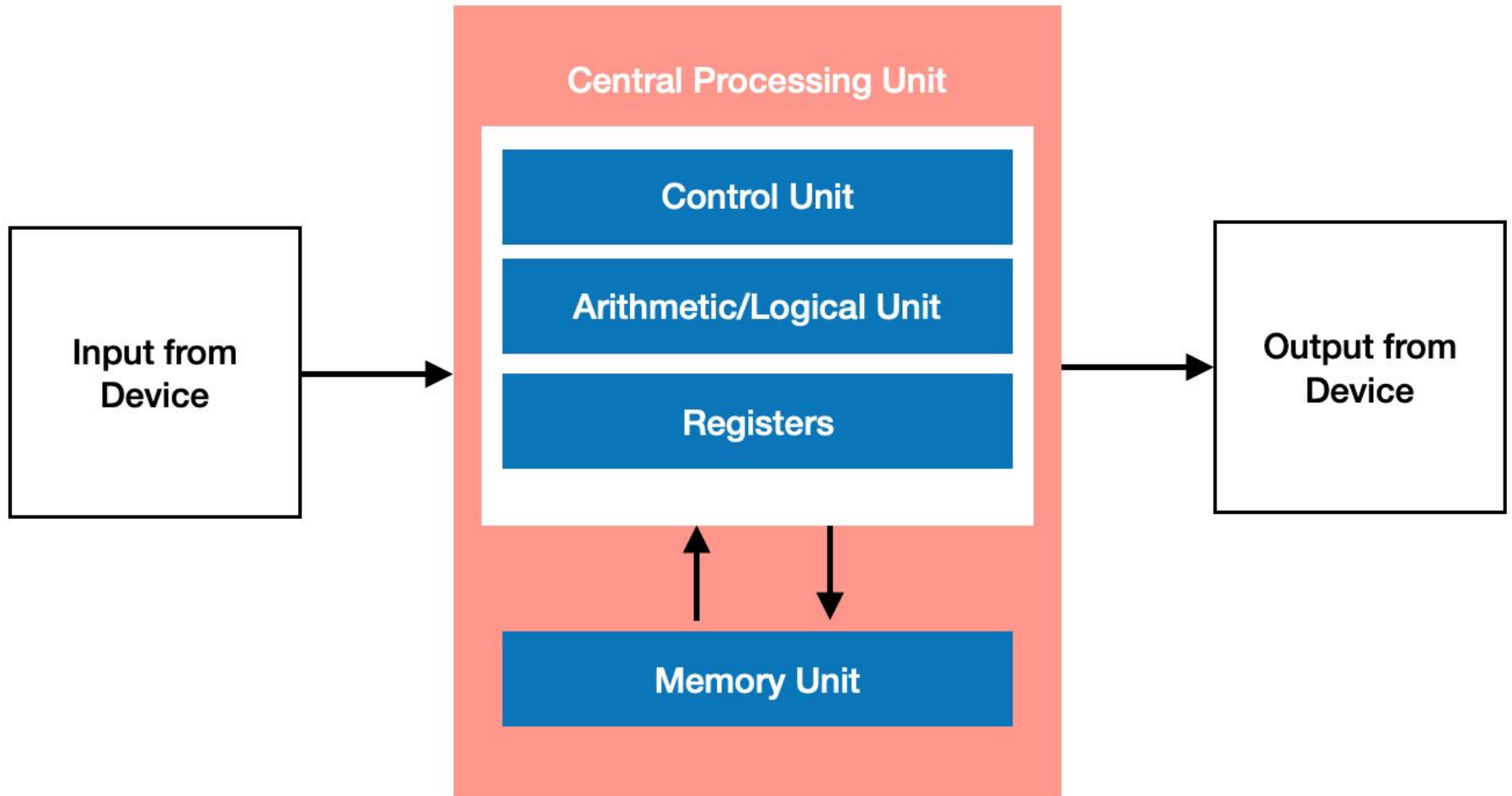


7. Sicurezza del Software

Sicurezza dell'Informazione

Architettura Von Neumann



Codice macchina (o assembly)

- Codice che permette di programmare direttamente il processore.
- Consente di utilizzare la memoria (RAM) e i registri per eseguire operazioni sui dati.
- Ogni istruzione corrisponde a un'operazione semplice del processore (carica dalla memoria, somma, sottrai, ecc.).

```
add rdi, rsi  
sub rsi, rdx  
mov rsi, [rdi]  
mov [rax], rdx
```

<https://x64.halb.it/>

Linguaggi di programmazione

- Scrivere codice macchina è scomodo:
 - E' poco intuitivo.
 - E' difficile da ottimizzare.
 - Il contesto è troppo ampio.
- Abbiamo sviluppato linguaggi di programmazione per poter scrivere del codice in una maniera più “intuitiva”.
- Da questo codice, anche chiamato sorgente, un **compilatore** (traduttore) traduce da linguaggio di programmazione a codice macchina.

Hello World

Primo programma in C. Il file si chiama main.c

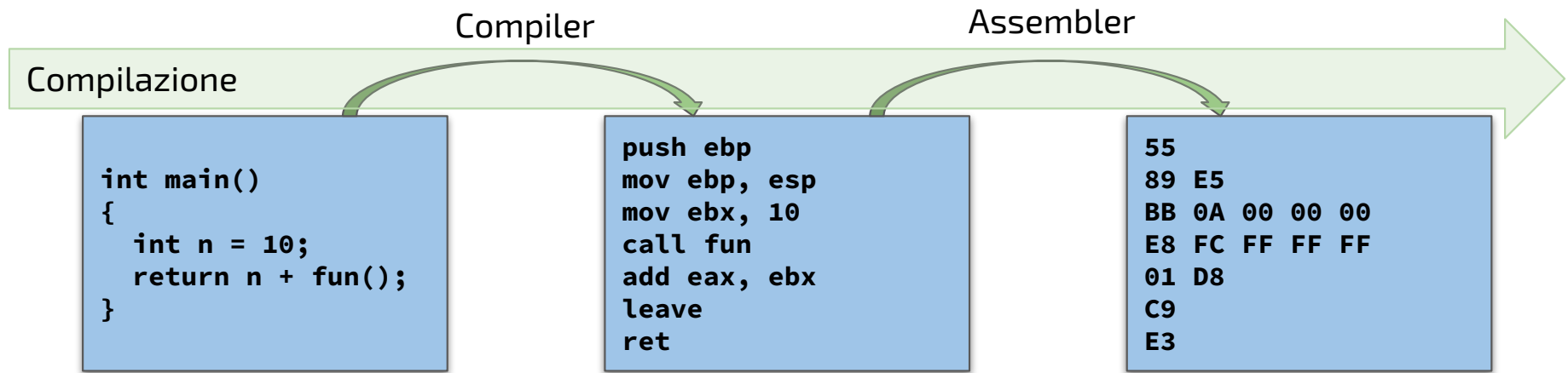
```
int main() {  
    printf("Hello world!\n");  
}
```

main() e **printf()** sono delle funzioni!

Si compila con:

```
gcc main.c -o main
```

Compilazione



Programma vulnerabile: Buffer overflow

```
int main() {
    bool admin = false;
    char username[16];
    char access_token[16];

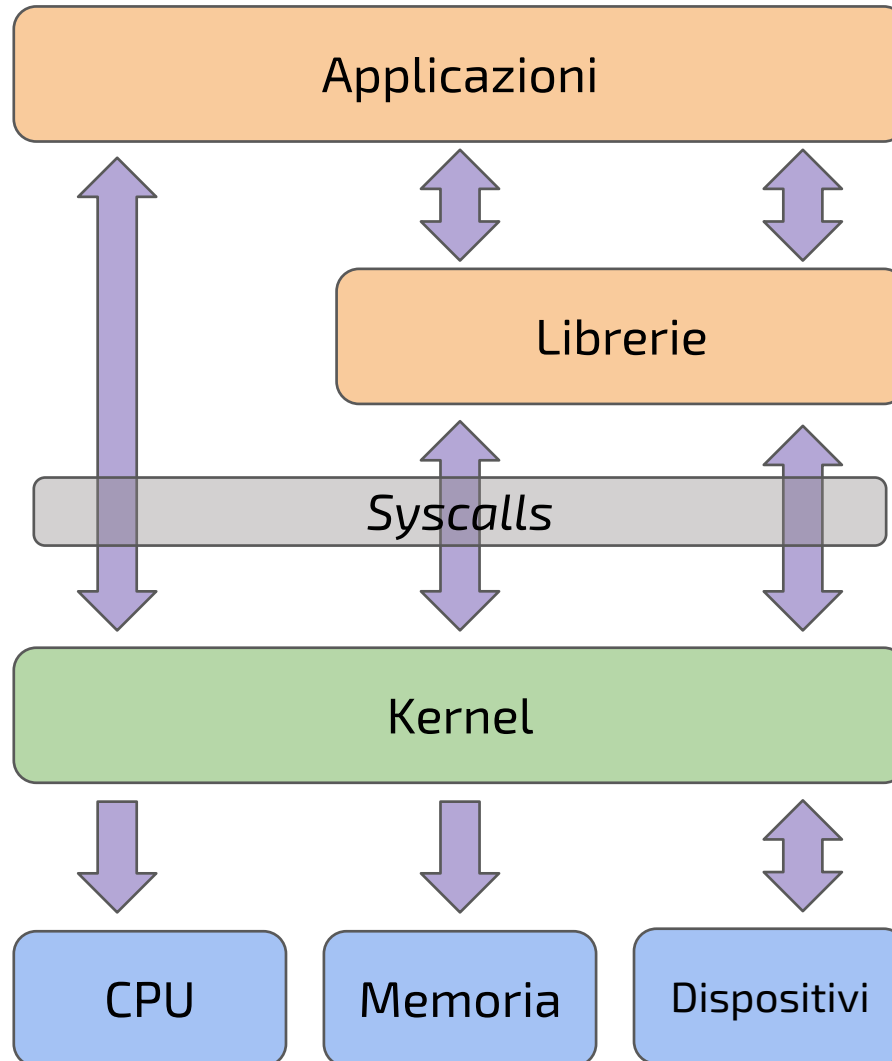
    printf("Access Token: ");
    scanf("%s", access_token);
    if (strcmp("H1XSD4545", access_token) == 0)
        admin = true;
    printf("Username: ");
    scanf("%s", username);
    if (admin == true)
        printf("Welcome admin!\n");
    else
        printf("Sorry %s, access denied!\n", username);
}
```

Vulnerabilità nel software

Come per le applicazioni web, anche nel software la mancata sanitizzazione dell'input crea problemi, ma esistono anche altre classi di vulnerabilità più importanti. In particolare:

- **Vulnerabilità spaziali:**
(legate all'accesso fuori dai limiti della memoria)
→ es. buffer overflow, out-of-bounds write/read
- **Vulnerabilità temporali**
(legate all'uso scorretto della memoria nel tempo)
→ es. use-after-free, double free, dangling pointers
- **Vulnerabilità di inizializzazione**
(uso di memoria non inizializzata)
→ es. use of uninitialized variables, memory disclosure

Applicazioni e Sistema Operativo



Buona ingegneria del software

Buona ingegneria del software: soddisfare i requisiti

- Requisiti funzionali
 - Il software deve fare ciò per cui è stato progettato.
- Requisiti non funzionali
 - Usabilità
 - Sicurezza (safety)
 - Sicurezza informatica (security)
- Creare applicazioni intrinsecamente sicure è una competenza fondamentale — ma spesso poco conosciuta — per un buon sviluppatore o ingegnere del software.
 - Creare software sicuro è difficile.

Il software ha vulnerabilità

Il software dovrebbe implementare le specifiche.

- Una specifica non rispettata == *bug del software*
- Una specifica di sicurezza non rispettata == *vulnerabilità*

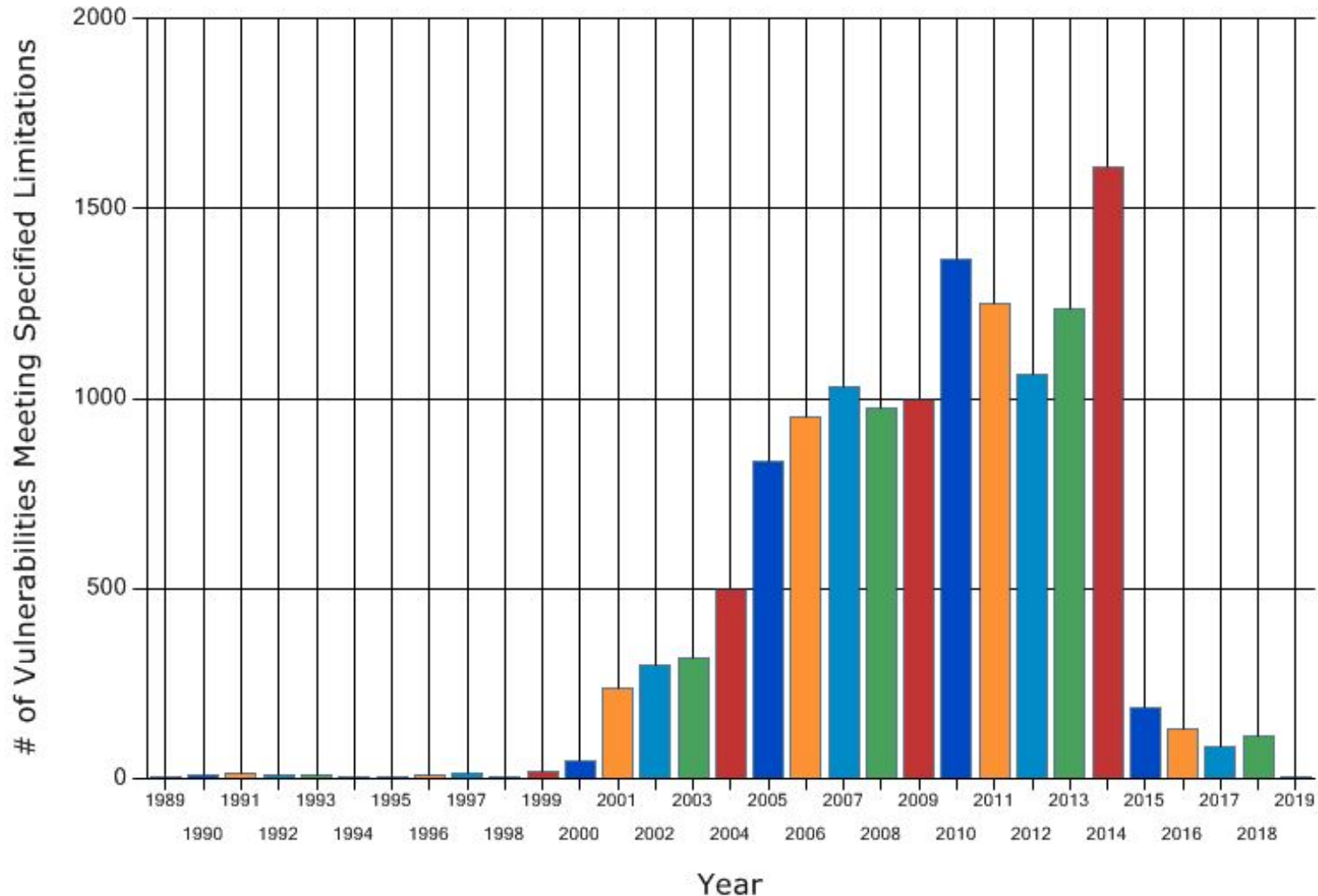
Un modo per sfruttare una vulnerabilità per violare la CIA (Confidentiality, Integrity, Availability) si chiama exploit.

- Vulnerabilità $\neq$ exploit.

Vita di una vulnerabilità Software

Vulnerabilità negli anni

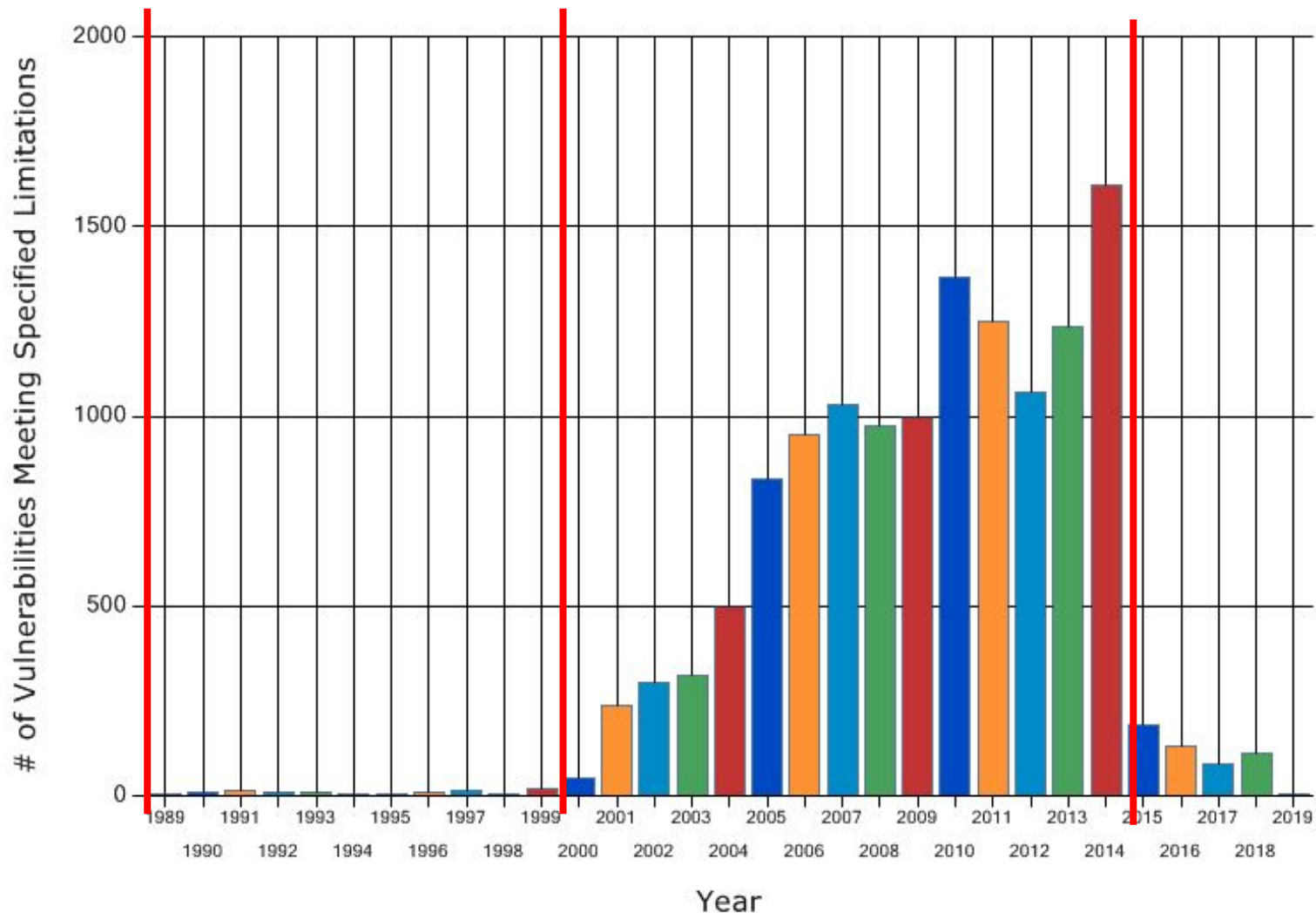
Total Matches By Year



Source: [NIST' National Vulnerability Database](#)

Vulnerabilità negli anni

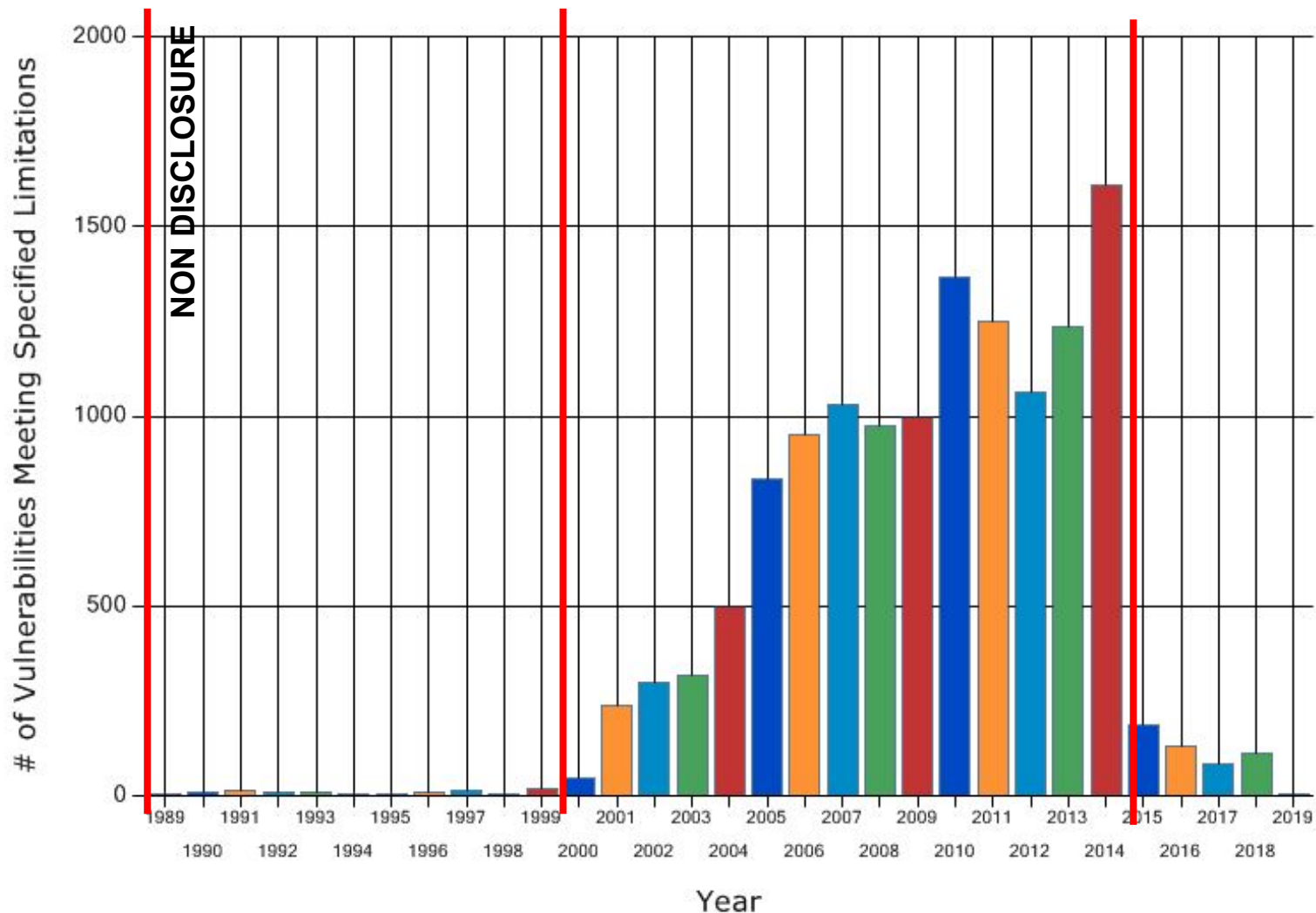
Total Matches By Year



Source: [NIST' National Vulnerability Database](#)

Vulnerabilità negli anni

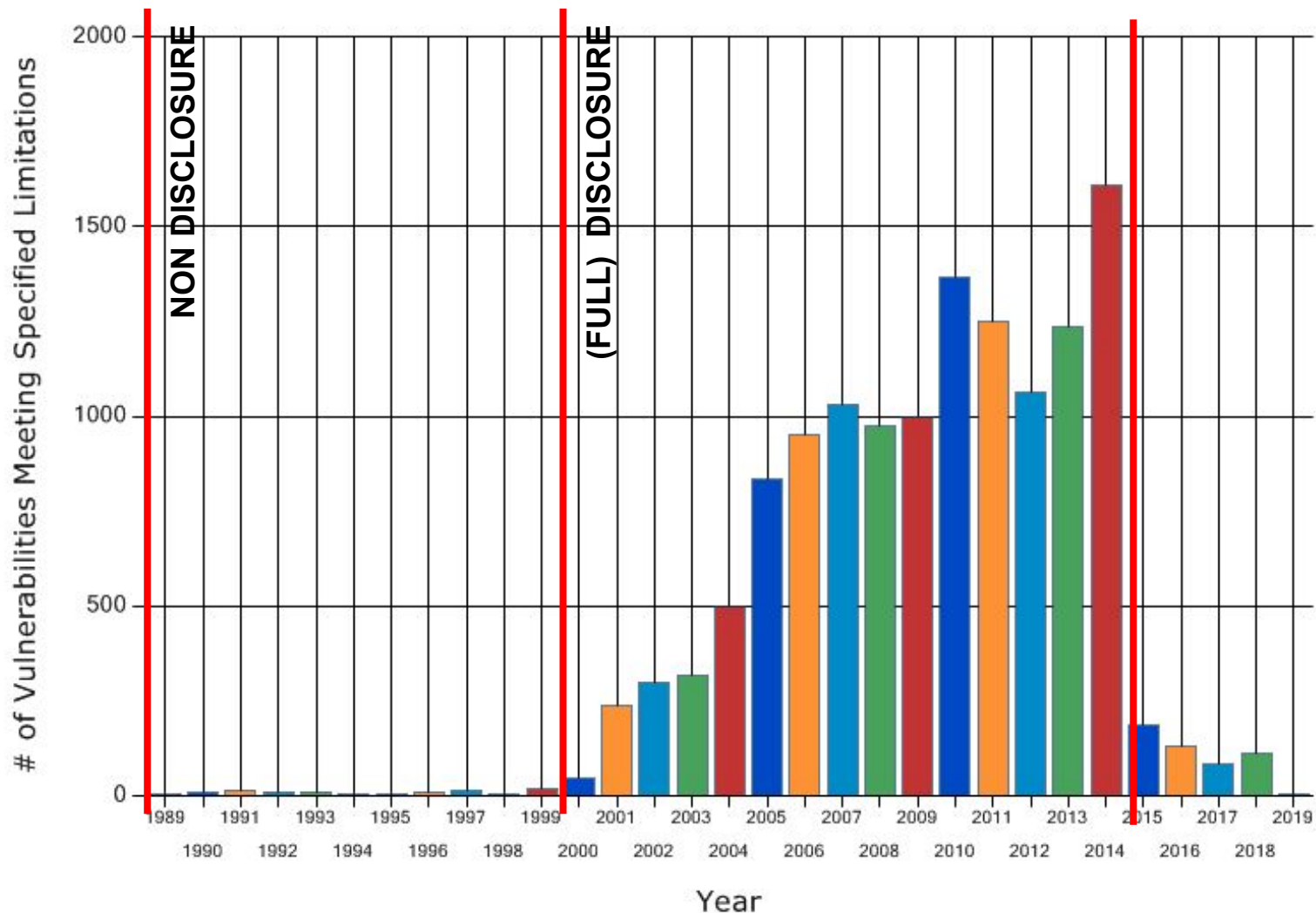
Total Matches By Year



Source: [NIST' National Vulnerability Database](#)

Vulnerabilità negli anni

Total Matches By Year



Source: [NIST' National Vulnerability Database](#)

I primi anni della divulgazione delle vulnerabilità

Subject: Comments on the dvwssr.dll vulnerability threads

From: Iván Arce

Date: 2000-04-18 1:25:52

I do not intend to go further down the full disclosure vs. mediated release of information discussion here, however [Microsoft's handler's] post on NTBugtraq regarding CORE's work requires some clarifications on our side.

[...]

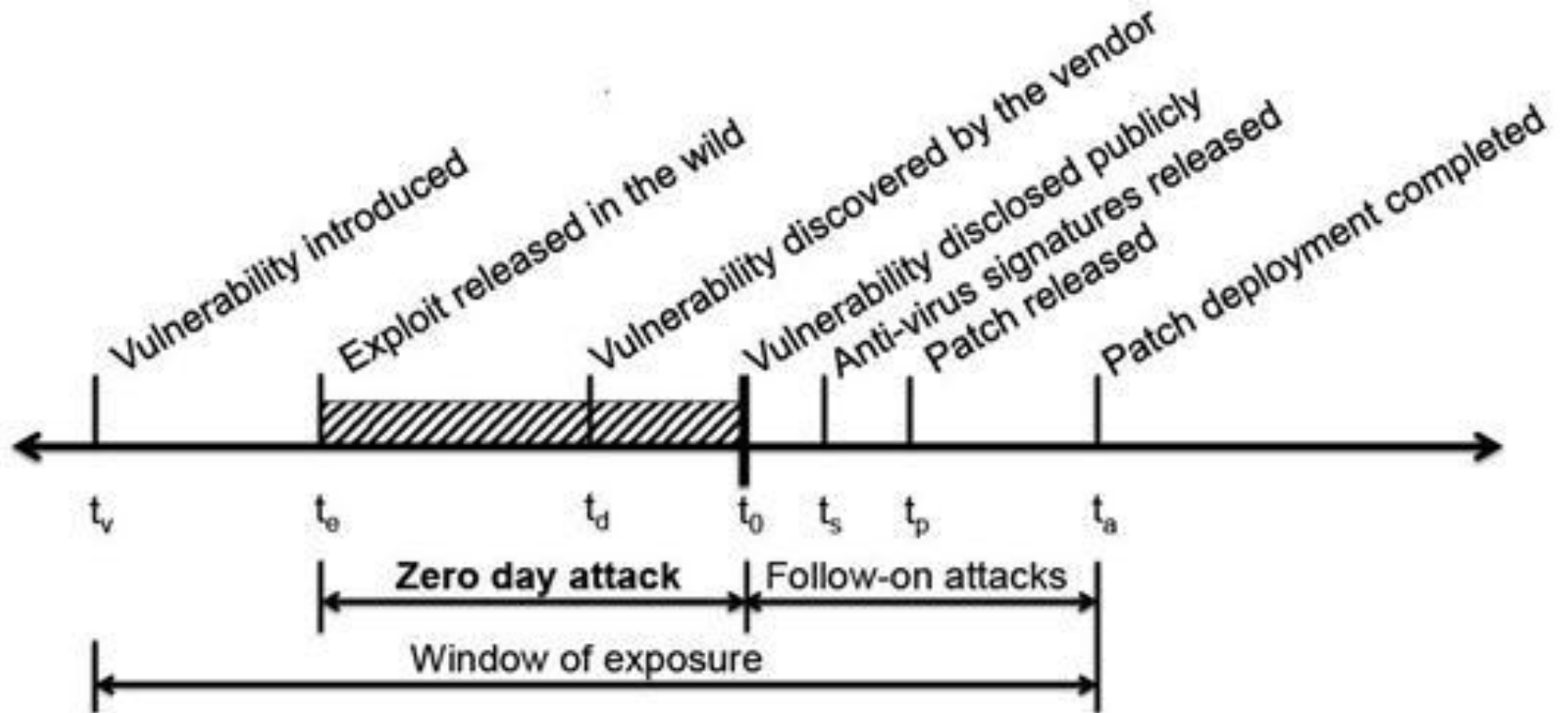
If someone yells 'FIRE' and that appears to be reasonable, I'd would be very careful in my methodology and editorial policies before yelling "NOT TRUE! NOT TRUE! EVERYTHING IS FINE!".

[...]

Excuse me if I'm being rude, but **I'm shocked by the fact that our company is being questioned because we found a bug.**

<http://www.s0ftpj.org/bfi/online/bfi10/BFi10-02.html>
<http://marc.info/?l=vuln-dev&m=95602682515862&w=2>

Ciclo di vita di una vulnerabilità

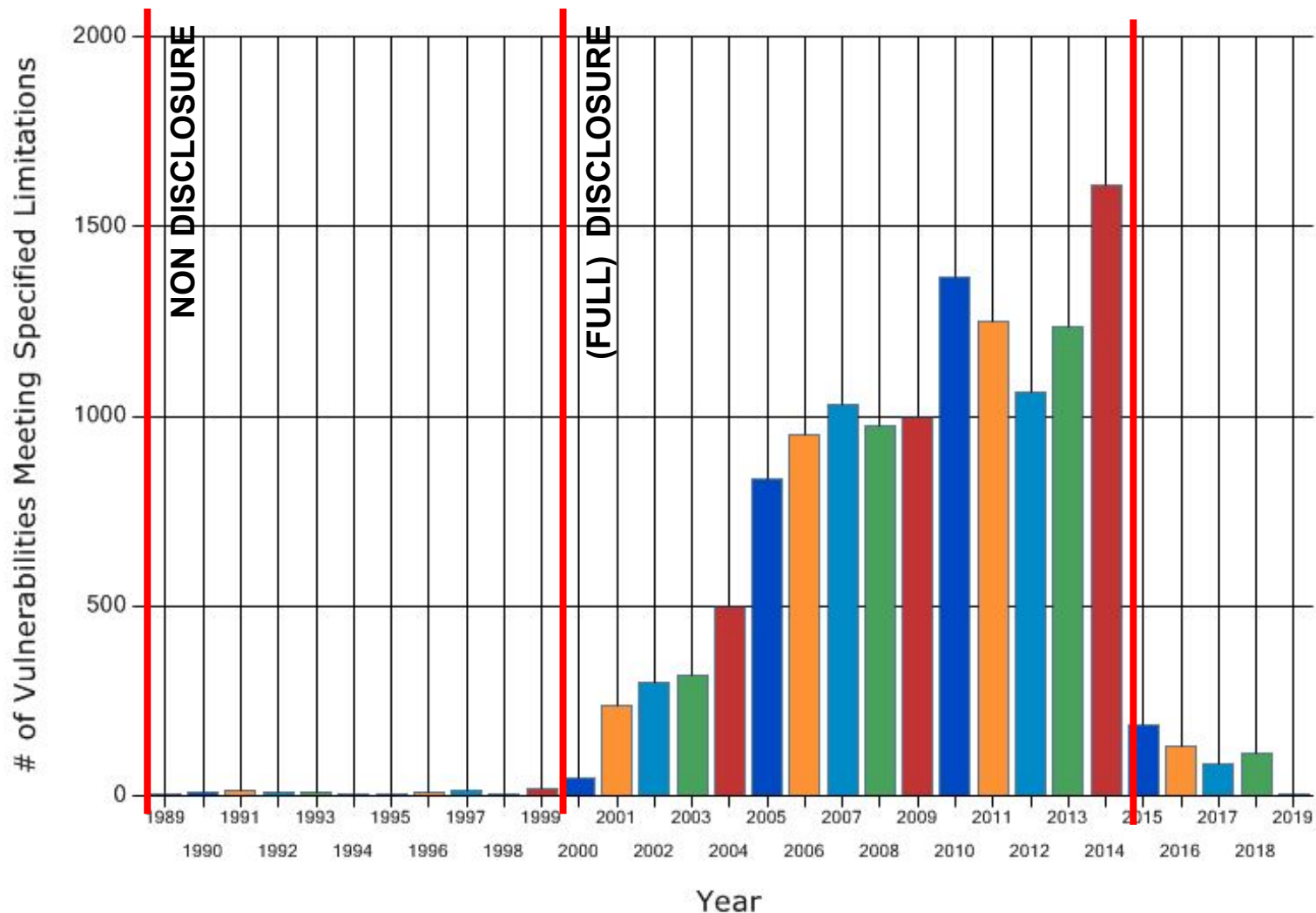


Black Hat



Vulnerabilità negli anni

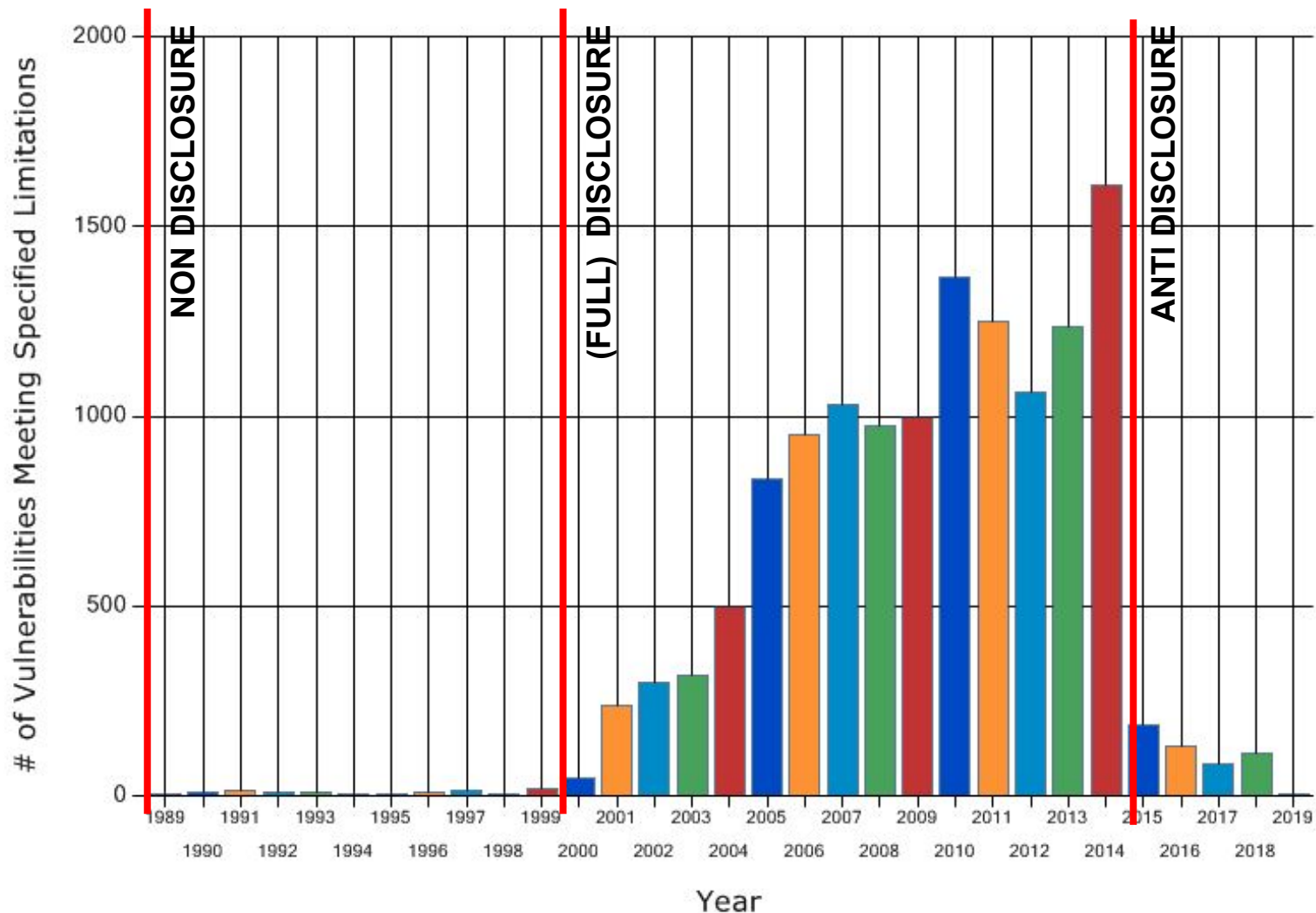
Total Matches By Year



Source: [NIST' National Vulnerability Database](#)

Vulnerabilità negli anni

Total Matches By Year



Source: [NIST' National Vulnerability Database](#)



Il black market degli exploit



Bug Bounties

Build the Next Security Defense Technology and You Could Win \$200,000

WHY ARE WE DOING THIS?

The Microsoft BlueHat Prize contest is designed to generate new ideas for defensive approaches to support computer security. As part of our commitment to a more secure computing experience, we hope to inspire security researchers to develop innovative solutions intended to address serious security threats.

WHAT IS THE CONTEST?

The inaugural Microsoft BlueHat Prize contest challenges security researchers to design a novel runtime mitigation technology designed to prevent the exploitation of memory safety vulnerabilities. The solution considered to be the most innovative by the Microsoft BlueHat Prize board will be presented the grand prize of US \$200,000. Important information:

- Entries will be accepted and must be received by email to bluehatprize@microsoft.com between August 3rd 2011 to midnight Pacific Time on April 1st 2012.
- The winning entry will be announced at Black Hat USA 2012.
- For full details, see rules and regulations.

YOU COULD WIN

First prize: \$200,000 (USD)
Second prize: \$50,000 (USD)
Third prize: MSDN Universal subscription valued at \$10,000 (USD)

QUESTIONS?

Send your questions or comments to bluehatprize@microsoft.com.

EMBED EMAIL SHARE INFORMATION



00:12 / 01:25

HOW DO I ENTER?

To enter, send an email to bluehatprize@microsoft.com — include your technical description and prototype as outlined in the official rules.

The Microsoft BlueHat Prize board will reply with additional information applicants will need to submit a complete entry.

 **SUBMIT EMAIL**

BlueHat Prize

 Tweet 150

 Recommend 43

© 2011 Microsoft Corporation. All Rights Reserved. Terms of Use | Privacy Statement | Trademarks

Altri Bug Bounties

All

219 ▾

Q sort:promoted-desc

219 results matching search - Find charity programs using charity:true

Recent



Kistler Vulnerability Disclosure Program
Measure. Analyze. Innovate.

- 🚩 Points per vulnerability
- 🔒 Safe harbor
- 🏆 Managed by Bugcrowd

[Submit report](#)



Recent



Better
Find out how much a better mortgage might save you.

🚩 \$200 - \$10,000 per vulnerability

- 🔒 Safe harbor
- 🏆 Managed by Bugcrowd

[Submit report](#)



Seeking specialists
We're looking for researchers to work on select private progr...

[Learn more](#)

Recent

Jora

Jora
Help Secure Jora

- 🚩 \$50 - \$2,500 per vulnerability
- 🔒 Partial safe harbor
- 🏆 Managed by Bugcrowd

[Submit report](#)



Recent



Opera Public Bug Bounty
Opera is a leading global internet brand with a large, engage...

🚩 \$200 - \$4,000 per vulnerability

- 🔒 Safe harbor
- 🏆 Managed by Bugcrowd

[Submit report](#)



ZERODIUM Payouts for Desktops/Servers*

Windows
macOS
Linux/BSD
Any OS

RCE: Remote Code Execution
LPE: Local Privilege Escalation
SBX: Sandbox Escape or Bypass
VME: Virtual Machine Escape

Up to \$1,000,000										1.001 Win RCE Zero Click Win
Up to \$500,000							3.001 Chrome RCE+LPE Win	2.001 Apache RCE Linux	2.002 MS IIS RCE Win	
Up to \$250,000						5.001 MS Outlook RCE Win	4.001 MS Exchange RCE Win	2.003 OpenSSL RCE Linux	2.004 PHP RCE Linux	
Up to \$200,000	6.001 VMware ESXi VME Win/Linux	5.002 Thunderbird RCE Win/Linux			4.002 Sendmail RCE Linux	4.003 Postfix RCE Linux	4.004 Dovecot RCE Linux	4.005 Exim RCE Linux	2.005 nginx RCE Linux	
Up to \$100,000		3.002 Safari RCE+LPE Mac	3.003 Edge RCE+LPE Win	3.004 Firefox RCE+LPE Win	5.003 Word/Excel RCE Win	7.001 WordPress RCE Linux	7.002 cPanel/WHM RCE Linux	7.003 Plesk RCE Linux	7.004 Webmin RCE Linux	
Up to \$80,000	6.002 VMware WS VME Win/Linux					5.004 Adobe PDF RCE+SBX Win	5.005 WinRAR RCE Win	5.006 7-Zip RCE Win	6.003 Windows LPE/SBX Win	
Up to \$50,000	6.004 USB LPE Win/Mac	8.001 Antivirus RCE Win			5.007 WinZip RCE Win	5.008 tar RCE Linux	6.005 macOS LPE/SBX Mac	6.006 Linux LPE Linux	6.007 BSD LPE BSD	
Up to \$10,000	9.001 Routers RCE	8.002 Antivirus LPE Win	7.005 phpBB RCE Linux	7.006 vBulletin RCE Linux	7.007 MyBB RCE Linux	7.008 Joomla RCE Linux	7.009 Drupal RCE Linux	7.010 Roundcube RCE Linux	7.011 Horde RCE Linux	

ZERODIUM Payouts for Mobiles*

Up to
\$2,500,000

Up to
\$2,000,000

Up to
\$1,500,000

Up to
\$1,000,000

Up to
\$500,000

Up to
\$200,000

Up to
\$100,000

FCP: Full Chain with Persistence
RCE: Remote Code Execution
LPE: Local Privilege Escalation
SBX: Sandbox Escape or Bypass

■ IOS
■ Android
■ Any OS

1.001
Android FCP
Zero Click
Android

1.002
iOS FCP
Zero Click
IOS

2.001
WhatsApp
RCE+LPE
Zero Click
IOS/Android

2.002
iMessage
RCE+LPE
Zero Click
IOS

2.003
WhatsApp
RCE+LPE
IOS/Android

2.004
SMS/MMS
RCE+LPE
IOS/Android

3.001
Persistence
IOS

2.005
WeChat
RCE+LPE
IOS/Android

2.006
iMessage
RCE+LPE
IOS

2.007
FB Messenger
RCE+LPE
IOS/Android

2.008
Signal
RCE+LPE
IOS/Android

2.009
Telegram
RCE+LPE
IOS/Android

2.010
Email App
RCE+LPE
IOS/Android

4.001
Chrome
RCE+LPE
Android

4.002
Safari
RCE+LPE
IOS

5.001
Baseband
RCE+LPE
IOS/Android

6.001
LPE to
Kernel/Root
IOS/Android

2.011
Media Files
RCE+LPE
IOS/Android

2.012
Documents
RCE+LPE
IOS/Android

4.003
SBX
for Chrome
Android

4.004
Chrome RCE
w/o SBX
Android

4.005
SBX
for Safari
IOS

4.006
Safari RCE
w/o SBX
IOS

7.001
Code Signing
Bypass
IOS/Android

5.002
WiFi
RCE
IOS/Android

5.003
RCE
via MitM
IOS/Android

6.002
LPE to
System
Android

8.001
Information
Disclosure
IOS/Android

8.002
[k]ASLR
Bypass
IOS/Android

9.001
PIN
Bypass
Android

9.002
Passcode
Bypass
IOS

9.003
Touch ID
Bypass
IOS

Panoramica sulla Raccolta e Categorizzazione delle Vulnerabilità

Come vengono raccolte le vulnerabilità:

- Divulgazione pubblica.
- Programmi di Bug Bounty.
- Scansione automatizzata.
- Risposta agli incidenti.

Categorizzazione delle vulnerabilità:

- **CVE (Common Vulnerabilities and Exposures)**: Identificatore univoco e descrizione dettagliata.
- **CWE (Common Weakness Enumeration)**: Classifica le vulnerabilità.
- **CVSS (Common Vulnerability Scoring System)**: Valuta la gravità in base a sfruttabilità e impatto.
- **CAPEC (Common Attack Pattern Enumeration and Classification)**: Documenta i metodi di attacco comuni.

Strumenti e piattaforme moderni:

- **National Vulnerability Database (NVD)**
- **MITRE ATT&CK Framework**
- Database di vulnerabilità/exploit come **VulnDB**, **Exploit-DB**

Exploitation Roadmap

1. Identificare le vulnerabilità:
 - a. Leggere il codice sorgente/Reverse engineering.
 - b. Fuzzing.
 - c. Modelli di Machine Learning/Large Language Models (GPT).
2. Capire le potenzialità delle vulnerabilità (alcune non sono sufficienti per l'exploitation).
3. Identificare la sequenza di input esatti per ottenere il comportamento desiderato.

Reverse Engineering

```
int main()
{
    int n = 10;
    return n + fun();
}
```

```
push ebp
mov ebp, esp
mov ebx, 10
call fun
add eax, ebx
leave
ret
```

```
55
89 E5
BB 0A 00 00 00
E8 FC FF FF FF
01 D8
C9
E3
```

Decompiler

Disassembler

Reverse Engineering

Cyber Resilience Act (CRA) - 2024

È oggi la legge europea più rilevante perché introduce, per la prima volta, il **riconoscimento formale della ricerca di vulnerabilità**. Prevede:

- Riconoscimento esplicito della **vulnerability disclosure in buona fede**.
- Obbligo per i produttori di avere una **politica di divulgazione delle vulnerabilità (VCDP)**.
- Le aziende **non possono perseguire o ostacolare** chi:
 - analizza un prodotto,
 - rileva vulnerabilità,
 - le comunica responsabilmente al produttore.
- Tuttavia:
 - **Non è un vero safe harbor penale**: ogni stato UE mantiene le sue leggi penali.
 - Non protegge attività che comportano **accesso non autorizzato a sistemi altrui**.

Conclusioni

La sicurezza informatica **non è un requisito funzionale** per molte realtà. Di conseguenza, il codice scritto non sarà mai sicuro.

Un software senza vulnerabilità non esiste.

Molte non sono state ancora identificate, molte verranno usate come 0-day, molte non verranno identificate.